34.1 日志系统

> **生成模型**：Claude Opus 4.6 (anthropic/claude-opus-4-6)
> **Token 消耗**：输入 ~48k tokens，输出 ~6k tokens（本节）

日志系统是 OpenClaw 运行时的"黑匣子"——它记录 Gateway 的每一次请求、每一个通道的连接状态、每一次 Agent 调用的来龙去脉。OpenClaw 基于 tslog 库构建了一套双轨日志架构：文件日志（结构化 JSON）用于事后分析，控制台日志（彩色文本）用于实时监控。

34.1.1 日志框架：tslog（`src/logger.ts`）

tslog 简介

> **衍生解释 — tslog**
>
> tslog 是一个 TypeScript 原生的日志库，特点是零依赖、类型安全、支持结构化日志。与 winston、pino 等流行日志库不同，tslog 的设计理念是"所见即所得"——日志对象的字段类型在编译期就已确定。
> OpenClaw 使用 tslog 的 `hidden` 模式（不输出 ANSI 格式），通过自定义 transport 将日志写入文件。

日志分级

OpenClaw 定义了七个日志级别，按严重程度从高到低排列：

```
// src/logging/levels.ts
export const ALLOWED_LOG_LEVELS = [
  "silent",  // 不输出任何日志
  "fatal",   // 致命错误（进程将退出）
  "error",   // 错误
  "warn",    // 警告
  "info",    // 信息（默认级别）
  "debug",   // 调试（verbose 模式下启用）
  "trace",   // 追踪（最详细）
] as const;
```

tslog 内部使用数字表示级别：`fatal=0, error=1, warn=2, info=3, debug=4, trace=5`。`silent` 映射为 `+∞`，意味着没有任何日志能通过这个阈值。

Logger 工厂与滚动日志

`getLogger()` 是获取文件日志实例的统一入口，采用懒初始化 + 缓存模式：

```
// src/logging/logger.ts（简化）
export const DEFAULT_LOG_DIR = "/tmp/openclaw";

export function getLogger(): TsLogger<LogObj> {
  const settings = resolveSettings();
  const cachedLogger = loggingState.cachedLogger;
  // 设置变化时重建 logger
  if (!cachedLogger || settingsChanged(cachedSettings, settings)) {
    loggingState.cachedLogger = buildLogger(settings);
    loggingState.cachedSettings = settings;
  }
  return loggingState.cachedLogger;
}
```

日志文件采用**按日滚动**策略，文件名格式为 `openclaw-YYYY-MM-DD.log`：

```
function defaultRollingPathForToday(): string {
  const today = formatLocalDate(new Date()); // "2026-02-18"
  return path.join(DEFAULT_LOG_DIR, `openclaw-${today}.log`);
}
```

每次创建新的 Logger 时，自动清理超过 24 小时的旧日志文件：

```
function pruneOldRollingLogs(dir: string): void {
  const cutoff = Date.now() - MAX_LOG_AGE_MS; // 24h
  for (const entry of fs.readdirSync(dir)) {
    if (entry.name.startsWith("openclaw-") && entry.name.endsWith(".log")) {
      const stat = fs.statSync(fullPath);
      if (stat.mtimeMs < cutoff) {
        fs.rmSync(fullPath, { force: true });
      }
    }
  }
}
```

> 日志目录固定在 `/tmp/openclaw`（macOS 上 `os.tmpdir()` 可能返回用户随机路径，导致 Debug UI 的"打开日志"按钮失效）。

文件日志 Transport

每条日志以 JSON 格式追加到文件：

```
logger.attachTransport((logObj: LogObj) => {
  const time = logObj.date?.toISOString?.() ?? new Date().toISOString();
  const line = JSON.stringify({ ...logObj, time });
  fs.appendFileSync(settings.file, `${line}\n`, { encoding: "utf8" });
});
```

日志行示例：


```
{"0":"gateway started","_meta":{"name":"openclaw","date":"2026-02-18T08:30:00.123Z"},"time":"2
```


外部 Transport 扩展

日志系统支持注册外部 Transport，允许插件将日志转发到第三方服务：

```
export function registerLogTransport(transport: LogTransport): () => void {
  externalTransports.add(transport);
  // 如果 logger 已存在，立即附加
  if (loggingState.cachedLogger) {
    attachExternalTransport(loggingState.cachedLogger, transport);
  }
  // 返回注销函数
  return () => { externalTransports.delete(transport); };
}
```

Transport 回调中的异常被静默吞掉——**日志永远不能阻塞主流程**。

Pino 兼容适配器

WhatsApp 库 Baileys 期望使用 pino 风格的日志接口。 `toPinoLikeLogger` 将 tslog Logger 适配为 pino 接口：

```
export function toPinoLikeLogger(logger: TsLogger<LogObj>, level: LogLevel): PinoLikeLogger {
  return {
    level,
    child: (bindings) => toPinoLikeLogger(logger.getSubLogger({ name: JSON.stringify(bindings)
    trace: (...args) => logger.trace(...args),
    debug: (...args) => logger.debug(...args),
    // ... info, warn, error, fatal
  };
}
```

34.1.2 日志分级与输出（src/logging/ 目录）

双轨输出架构

OpenClaw 的日志输出分为两条独立的轨道：

```
应用代码
    │
    ├── 文件轨道（File Logger）
    │    ├── 级别过滤：由 logging.level 配置控制（默认 info）
    │    ├── 格式：JSON，每行一条
    │    └── 目标：/tmp/openclaw/openclaw-YYYY-MM-DD.log
    │
    └── 控制台轨道（Console Logger）
         ├── 级别过滤：由 logging.consoleLevel 配置控制（verbose 模式下自动提升到 debug）
         ├── 格式：三种样式 pretty / compact / json
         └── 目标：stdout（或 stderr，RPC 模式下）
```

控制台输出样式

样式	触发条件	格式
pretty	TTY 终端（默认）	08:30:00 [discord] 彩色消息
compact	非 TTY 或显式配置	2026-02-18T08:30:00.123Z [discord] 消息
json	显式配置	{"time":"...","level":"info","subsystem":"discord","message":"..."}

子系统日志（Subsystem Logger）

OpenClaw 的子系统日志是日志系统最精巧的部分——它为每个模块（如 `discord` 、 `telegram` 、 `gateway/ws` ）创建独立的带前缀日志实例：

```
// src/logging/subsystem.ts（简化）
export function createSubsystemLogger(subsystem: string): SubsystemLogger {
  const emit = (level: LogLevel, message: string, meta?: Record<string, unknown>) => {
    // 1. 写入文件（总是写入，受文件级别过滤）
    logToFile(getFileLogger(), level, message, meta);

    // 2. 控制台级别过滤
    if (!shouldLogToConsole(level, consoleSettings)) return;

    // 3. 子系统过滤器（可配置只显示特定子系统）
    if (!shouldLogSubsystemToConsole(subsystem)) return;

    // 4. 格式化并输出
    const line = formatConsoleLine({ level, subsystem, message, style });
    writeConsoleLine(level, line);
  };

  return {
    subsystem,
    trace: (msg, meta) => emit("trace", msg, meta),
    debug: (msg, meta) => emit("debug", msg, meta),
    info:  (msg, meta) => emit("info",  msg, meta),
    warn:  (msg, meta) => emit("warn",  msg, meta),
    error: (msg, meta) => emit("error", msg, meta),
    fatal: (msg, meta) => emit("fatal", msg, meta),
    raw: (msg) => { /* 直接输出，不格式化 */ },
    child: (name) => createSubsystemLogger(`${subsystem}/${name}`),
  };
}
```

子系统名称支持层级结构（ `"gateway/ws"` 、 `"agent/embedded"` ）， `child()` 方法创建子日志实例。

子系统颜色分配

控制台输出中，每个子系统使用不同的颜色以便区分：

```
const SUBSYSTEM_COLORS = ["cyan", "green", "yellow", "blue", "magenta", "red"] as const;

function pickSubsystemColor(color: ChalkInstance, subsystem: string): ChalkInstance {
  // 特定子系统有固定颜色覆盖
  const override = SUBSYSTEM_COLOR_OVERRIDES[subsystem];
  if (override) return color[override];
  // 其余使用名称哈希值分配
  let hash = 0;
  for (let i = 0; i < subsystem.length; i++) {
    hash = (hash * 31 + subsystem.charCodeAt(i)) | 0;
  }
  return color[SUBSYSTEM_COLORS[Math.abs(hash) % SUBSYSTEM_COLORS.length]];
}
```

Console Capture — 控制台劫持

`enableConsoleCapture()` 是 CLI 启动流程中的关键一步——它劫持 `console.log/info/warn/error/debug/trace`，将所有控制台输出同时写入文件日志：

```typescript
export function enableConsoleCapture(): void {
  if (loggingState.consolePatched) return; // 幂等
  loggingState.consolePatched = true;

  const original = {
    log: console.log,
    info: console.info,
    // ...
  };

  const forward = (level: LogLevel, orig: Function) => (...args: unknown[]) => {
    const formatted = util.format(...args);
    if (shouldSuppressConsoleMessage(formatted)) return; // 抑制噪音

    // 写入文件日志
    getLoggerLazy()[level](formatted);

    // 写回控制台（或 stderr）
    if (loggingState.forceConsoleToStderr) {
      process.stderr.write(`${formatted}\n`);
    } else {
      orig.apply(console, args);
    }
  };

  console.log = forward("info", original.log);
  console.warn = forward("warn", original.warn);
  // ...
}
```

特殊处理：

- **噪音抑制**：非 verbose 模式下，`"Closing session:"`、`"Opening session:"` 等高频消息被静默
- **EPIPE 处理**：管道断裂（EPIPE）错误被静默吞掉，而不是导致进程崩溃
- **RPC/JSON 模式**：`routeLogsToStderr()` 将所有控制台输出重定向到 stderr，保持 stdout 干净（用于 `--json` 输出）
- **时间戳前缀**：可配置自动添加时间戳前缀（排除已有时间戳和 JSON 内容）

敏感信息脱敏

日志系统内置了一套正则表达式驱动的敏感信息脱敏引擎：

```ts
// src/logging/redact.ts
const DEFAULT_REDACT_PATTERNS: string[] = [
  // ENV-style: API_KEY=sk-abc123...
  String.raw`\b[A-Z0-9_]*(?:KEY|TOKEN|SECRET|PASSWORD)\b\s*[=:]\s*(["']?)([^\s"'\\]+)\1`,
  // JSON fields: "apiKey": "..."
  String.raw`"(?:apiKey|token|secret|password|accessToken)"\s*:\s*"([^"]+)"`,
  // CLI flags: --api-key sk-abc...
  String.raw`--(?:api[-_]?key|token|secret)\s+(["']?)([^\s"']+)\1`,
  // Bearer tokens
  String.raw`\bBearer\s+([A-Za-z0-9._\-+=]{18,})\b`,
  // Common prefixes: sk-..., ghp_..., xox-..., etc.
  String.raw`\b(sk-[A-Za-z0-9_-]{8,})\b`,
  String.raw`\b(ghp_[A-Za-z0-9]{20,})\b`,
  // ... 更多模式
];
```

脱敏策略：保留 Token 的前 6 位和后 4 位，中间用省略号替换：

```
sk-abc123456789xyz → sk-abc1…9xyz
```

PEM 私钥块保留首尾行，中间替换为 `…redacted…` 。

脱敏有两种模式： `"tools"` 模式（默认）只处理工具输出中的敏感信息； `"off"` 模式则完全关闭脱敏。

34.1.3 WebSocket 日志（ `src/gateway/ws-log.ts` ）

Gateway 的 WebSocket 通信会产生大量日志——每次 RPC 调用、每条消息广播、每个连接事件都需要记录。 `ws-log.ts` 提供了一套专门的 WebSocket 日志格式化工具。

日志格式化

`formatForLog()` 把任意值安全地转换为日志友好的字符串，长度限制在 240 字符以内，并自动脱敏：

```
export function formatForLog(value: unknown): string {
  // Error 对象：提取 name + message + code
  if (value instanceof Error) {
    return `${value.name}: ${value.message}${value.code ? ` code=${value.code}` : ""}`;
  }
  // 对象：提取 message 字段
  if (typeof value === "object" && value?.message) { ... }
  // 其他：JSON.stringify + 脱敏 + 截断
  const str = typeof value === "string" ? value : JSON.stringify(value);
  return redactSensitiveText(str, WS_LOG_REDACT_OPTIONS).slice(0, 240);
}
```

ID 缩写

WebSocket 消息里频繁出现的 UUID 会被缩写为 `前8位…后4位` 格式：

```
export function shortId(value: string): string {
  if (UUID_RE.test(value)) return `${value.slice(0, 8)}…${value.slice(-4)}`;
  if (value.length <= 24) return value;
  return `${value.slice(0, 12)}…${value.slice(-4)}`;
}
```

日志子系统

WebSocket 日志使用 `"gateway/ws"` 子系统标识，在控制台上显示为 `[ws]`（`gateway` 前缀会被自动剥离）。

本节小结

1. **tslog 基础**：OpenClaw 使用 tslog 作为日志框架，`hidden` 模式运行（不输出 ANSI），通过自定义 Transport 将 JSON 日志追加到文件
2. **七级日志**：silent → fatal → error → warn → info → debug → trace，默认级别 info，verbose 模式下控制台提升到 debug
3. **双轨输出**：文件轨道（JSON 格式、按日滚动、24h 自动清理）和控制台轨道（pretty/compact/json 三种样式、子系统颜色编码）
4. **子系统日志**：`createSubsystemLogger("discord")` 创建带前缀的日志实例，支持层级结构和子系统过滤，控制台颜色通过名称哈希分配
5. **Console Capture**：劫持全局 `console.*` 方法，同时写入文件日志，支持噪音抑制、EPIPE 容错和 stderr 重定向
6. **敏感信息脱敏**：正则表达式引擎自动识别 API Key、Token、密码等敏感字段，保留首尾字符并替换中间部分

7. **WebSocket 日志**：专用格式化工具限制输出长度、缩写 UUID、自动脱敏，使用 `gateway/ws` 子系统标识